SPEC — CRM (Admin customer console)

SECTION 06-operations-it TYPE spec STATUS draft OWNER Product UPDATED 2026-08-19

Status: Draft · **Owner:** Product · **Last updated:** 2026-08

An internal, staff-only console for Fly GACA to see and manage the people and organizations on the platform — pilot users, their subscriptions, support threads, and the B2B pipeline of flight schools — in one place, instead of across a `psql` prompt, the Moyasar dashboard and an inbox.

> Internal & operational. This console is for Fly GACA staff only. It reads customer data the platform already holds (accounts, the `entitlements` row, billing status) under our Privacy Notice and PDPL obligations. It is **not** part of the public site, is `noindex`, and lives behind a staff-only gate on the API (see §5). It never exposes another user's data to a non-staff caller.

1. Why

Today, answering "who is this customer, what plan are they on, and what have they asked us?" means cross-referencing three tools by hand:

- **A `psql` session** against Cloud SQL for the account (`users` + `profiles`) and its plan (`entitlements`).
- **The Moyasar dashboard** for the payment and subscription state.
- **Email** (`i@flygaca.com`) for whatever they wrote in.

As Phase 5 (Money & flight schools) turns on paid plans and brings in organizations that buy seats, this gets worse fast. We need a single read-mostly surface that joins **account → entitlement → billing → support → pipeline**, so support is faster, churn is visible, and B2B leads don't fall through the cracks. The CRM is the operator's view of the customer; it reuses the data the rest of the product already generates rather than inventing a new source of truth.

Postgres helps here in a way the old Firestore model could not: account, plan, payment and progress all live in one database, so the customer view is a single join in `store.ts` rather than three fan-out reads stitched together in the client.

2. Who

- **Staff (admin)** — Fly GACA team. Staff identity already exists in `server/src/staff-core.ts`: `isStaffEmail(email, emailVerified)` matches a **verified** address against `STAFF_EMAILS` / `STAFF_DOMAINS` (`flygaca.com`), and `routes/grants.ts` uses it to issue the complimentary staff

entitlement. Today that predicate grants *access to the product*, not *operator authority* — see §5 for the gap this spec has to close. Primary and, in v1, only user of this console.

- **Sales/ops (future role)** — a narrower `crm` role for someone who works the pipeline but should not see, e.g., raw billing identifiers. Out of scope for v1; the data model leaves room for it.
- **Customer** — the subject of a record, not a user of the console. Either a **pilot** (a row in `users`) or, later, a **school**/org (`orgs`, `schools`).

3. Scope

In scope (v1)

- A **customer list**: every pilot account with at-a-glance plan, status, last active and signup date; searchable by email/name; filterable by plan and status (active / expiring / lapsed / free).
- A **customer detail** view that joins, read-only:
  - account basics from `users` + `profiles` (display name, email, locale, created),
  - the plan from the `entitlements` row (plan, source, expiry),
  - a billing summary from `payments` · `subscriptions` · `checkout_intents`, with a live Moyasar lookup only when the local rows are ambiguous — read server-side, never with a client-side Moyasar key,
  - the customer's support notes (below).
- **Internal notes**: staff can append timestamped notes to a customer (the one thing the CRM *writes*).
- A **B2B pipeline (leads)**: a lightweight kanban of flight-school prospects — stage, contact, value estimate, next action — feeding the "Fly GACA for Schools" engine in Phase 5.
- An **overview**: a few operational counters (active subs, entitlements expiring in 7 days, failed renewals, open leads).

Out of scope (v1)

- **Editing billing.** Refunds and cancellations stay in the Moyasar dashboard and the customer's own account page. The CRM links out; it does not mutate billing.
- **Editing the customer's account or entitlement** beyond notes. The `entitlements` table is written **only** by `routes/billing.ts` (checkout-config → confirm → webhook → the renewal job) and `routes/grants.ts` (staff · school-seat · founding) — the CRM must not become a second writer that drifts from billing.
- **Bulk email / marketing automation.** Transactional email stays in the billing flow; campaigns are a later, separate decision (Phase 8).
- **An external CRM integration** (HubSpot/Salesforce). Revisit only if the pipeline outgrows the in-house kanban.
- **Self-service org management** for school owners — that is the shipped `/business/admin` cohort dashboard, not this internal console.

4. Data model (Postgres)

Customer truth stays where it already lives — the account is `users` + `profiles`, the plan is the server-owned `entitlements` row, and billing state is `payments` / `subscriptions` / `checkout_intents`. The CRM adds only operator-owned tables, and they carry no foreign key into a customer's own data beyond `user_id`.

```
-- Both tables are operator data. No client ever reads them; only the
-- staff-gated router in §5 touches them.

CREATE TABLE crm_notes (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  subject_type text NOT NULL CHECK (subject_type IN ('user','lead')),
  subject_id   uuid NOT NULL,          -- users.id or crm_leads.id
  author_id    uuid NOT NULL REFERENCES users(id),   -- staff who wrote it
  body         text NOT NULL,
  created_at   timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX crm_notes_subject_idx ON crm_notes (subject_type, subject_id, created_at
DESC);

CREATE TABLE crm_leads (                -- B2B pipeline — flight schools / orgs
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_name          text NOT NULL,
  contact_name      text,
  contact_email     citext,
  phone             text,
  stage             text NOT NULL DEFAULT 'new'
                    CHECK (stage IN ('new','contacted','demo','proposal','won','lost')),
  seats_estimate    integer,
  value_estimate_sar integer,           -- SAR, whole riyals
  owner_id          uuid REFERENCES users(id),       -- staff owner of the deal
  next_action       text,
  next_action_at    timestamptz,
  source            text,               -- 'inbound' | 'referral' | 'event' | ...
  linked_org_id     uuid REFERENCES orgs(id),        -- set when the lead converts
  created_at        timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX crm_leads_stage_idx ON crm_leads (stage, next_action_at);
```

Read-only, **not** stored by the CRM:

Source	Owned by
`users`, `profiles`	the user (profile writes via `routes/account.ts`)
`entitlements`, `pack_entitlements`, `chat_credits`	the server — billing + grants only
`payments`, `subscriptions`, `checkout_intents`	`routes/billing.ts`
`orgs`, `org_seats`, `schools`, `school_invites`	`routes/grants.ts` + `routes/org.ts`

No customer-facing schema changes: the CRM adds two tables and reads the rest. A denormalized `crm_index` summary table for the customer list is an optional optimization and is deferred until the list is actually slow — with a real database behind it, that is much further out than it was under Firestore.

5. Security model

The mechanism changed with the Cloud Run port, and it changed in our favour: there is **no client-side database access at all**. The browser talks only to `/api/*`; every row it can see is one an Express route chose to return. What `firestore.rules` used to assert as a rule is now structural — a route that doesn't exist can't leak.

- **Every CRM read and write goes through a staff-gated Express router.** The console never queries Postgres directly; there is no path from the browser to `store.ts` except a route.
- **The gate itself is the one new primitive this spec needs.** `staff-core.ts` answers "is this a Fly GACA person?" but is used today only to *grant an entitlement*, and an entitlement is not an authorization role. v1 should add an explicit operator role rather than overload the entitlement:
  - a `role` column on `users` (`'user' | 'staff' | 'crm'`), written only by the `grant-staff-access.mjs` operational script, and
  - a `requireStaff(req)` middleware in `server/src/routes/crm.ts` that asserts it on every route, the way `requireUser(req)` asserts a session today. Gating on `isStaffEmail` alone would work for v1 (one operator, one domain) but ties operator authority to an email domain — see §7.
- Policy stays pure: the list/filter/pagination rules and the lead-stage transitions go in `server/src/crm-core.ts`, unit-tested like every other `*-core.ts`, with the router thin and all SQL in `store.ts`.
- **Billing identifiers** (Moyasar payment/subscription ids, card metadata) are read server-side and returned as a minimal summary; raw secrets and full payment data never reach the browser.
- **Auditability:** notes record `author_id` + `created_at`; lead stage changes stamp `updated_at` and `owner_id`. This is a PDPL-relevant surface — staff access to personal data should be attributable.
- The console is `noindex, nofollow` (via `useNoindexMeta`) and session-gated.

6. Surfaces

- A staff route in `src/router.tsx` — `/staff/crm`, lazy-loaded like every other page, behind the shared `Layout` chrome (never hand-edited; chrome is a component).
  - **Overview** — operational counters.
  - **Customers** — list + detail (account · entitlement · billing · notes).
  - **Pipeline** — leads kanban.

- `server/src/routes/crm.ts` (new), mounted under `/api/crm` in `server/src/index.ts` alongside `auth` · `account` · `grants` · `billing` · `org`:
  - `GET /api/crm/customers?query&plan&status&cursor`
  - `GET /api/crm/customers/:userId` — joins account + entitlement + billing summary
  - `POST /api/crm/notes` — `{ subjectType, subjectId, body }`
  - `GET /api/crm/leads` · `POST /api/crm/leads` · `PATCH /api/crm/leads/:id`
- Reuses `billing-core.ts` (`isActive` / plan derivation) and the existing billing tables for the plan/billing summary, rather than duplicating that logic. The client side reuses `src/lib/services/` conventions — one typed service module, no ad-hoc `fetch`.

7. Open questions

- **The operator role.** Add a `role` column (recommended) or gate on `isStaffEmail` for v1? The column is the honest primitive — access to the CRM and a free Pro plan are different things and should not share one predicate. Decide before milestone 1.
- **Customer search.** `ILIKE` on email/name is fine for the first few thousand accounts; a `tsvector` column on `profiles` is the next step, and neither needs a denormalized index table. (This was the hard problem under Firestore; it is not one now.)
- **A narrower `crm` (sales) role** distinct from `staff` — needed when a non-engineer works the pipeline. Defer until there's a second operator.
- **PDPL access logging.** Do we need to log *reads* of personal data, not just writes? Likely yes for staff access — confirm with the LAWYER-BRIEF scope. A `crm_access_log` table is cheap; the question is retention, not feasibility.
- **Leads ↔ orgs.** `crm_leads.linked_org_id` is in the schema above, but who sets it — the operator, by hand, on close-won, or `grant-org.mjs` when it provisions the org? Prefer the script, so the link can't be forgotten.

8. Milestones

1. **Console shell + staff gate.** Stand up `/staff/crm` behind `requireStaff`, with the `role` column and the `crm-core.ts` policy module. (No client-facing data access is added — the two new tables are reachable only through the router.)
2. **Customers (read-only).** List + detail joining `users` / `profiles`, `entitlements`, and the Moyasar billing summary from `payments` / `subscriptions`.
3. **Notes.** Append-only internal notes on a customer.
4. **Pipeline.** Leads kanban (stages, owner, next action) — the B2B on-ramp for "Fly GACA for Schools".
5. **Overview.** Operational counters (active / expiring / failed renewal / open leads).